GroundGate: Catching an Agent Before It Acts on a Bad Idea When the Evidence Is Imperfect

Perceptron Hackathon 2026 · Round 1 · Theme 6 (AI You Can Actually Trust) · Problem 9

Team: Neural Genesis · ID: TM-1EFBCA97D27 · Leader: Mouktika Jakkam · Members: Dharmi Sandip Sapariya

1. Problem and Theme Chosen

The costliest agent error is often a permitted, well-formed call resting on a false fact: a refund for an order that does not exist, belongs to another customer, or was already refunded. Nothing errors, so nothing flags it. Permission layers (AgentSpec, Progent, CaMeL) ask whether the agent *may* act, not whether its facts are *true*. We chose Problem 9 because grounding is where agent trust is won or lost, and our skills (agent frameworks, evaluation, backend security) fit it.

2. Understanding of the Challenge

Checking a call against "a verifiable source" sounds like a lookup. The difficulty is that evidence is imperfect and costly:

- **Absence is not contradiction.** A cache missing an order has not shown it is fabricated. Blocking on absence causes false blocks; trusting stale entries causes wrong executions.
- **Verification has a price.** Each authoritative lookup costs latency and budget, so the guard must decide *when* to pay.
- **Check-to-execute gap.** A verdict can be outdated by mutation time; retries and concurrent requests must not act twice.

3. Proposed Approach

Testbed: three mutating tools (refund, cancel, change address) over a SQLite system of record and an incomplete, possibly stale cache. The gate is standard-library Python; a LangGraph agent calls it.

1. **Typed action record:** validated arguments, reversibility, amount, idempotency key, and the premises the call depends on (order exists, belongs to this customer, status allows it, not already refunded).
2. **Evidence resolution:** observations carry source, authority and freshness. Authoritative beats cache; stale entries never support irreversible actions; free text is never evidence. Each premise becomes SUPPORTED, CONTRADICTED or UNVERIFIABLE.
3. **Verification cascade:** exact checks, then cache, then at most one read-only authoritative lookup under a budget, timeout and circuit breaker.
4. **Risk-table policy:** CONTRADICTED blocks; SUPPORTED allows; UNVERIFIABLE escalates if irreversible or high-value, otherwise allows with logging. A transparent rule table with assumed costs, not a calibrated model.
5. **Execution barrier:** the executor accepts only (tool, args, request_id), re-derives premises, risk and idempotency key, requires an expiring single-use action-bound token, then rechecks premises and applies a version-conditional update in one `BEGIN IMMEDIATE` transaction.

4. Minimum Viable Implementation

(1) A checker verifying refund calls against the system of record before execution. (2) A demo stopping a fabricated order ID, another customer's order, an already-refunded order, a forged token and a hostile free-text payload, while valid refunds pass. No LLM or GPU needed. This already runs: 81 automated tests pass (unit, security, multi-process race, benchmark integrity), and the benchmark reruns from a clean clone with 0 discrepancies. **Not yet built:** the LangGraph agent and any real-model run.

5. Stretch Goals

Risk-based routing: reversibility and amount already choose cache-only, one lookup, or escalation (rules, built). Next step: a **fast/careful router** where a slower LLM judge handles only UNVERIFIABLE high-risk calls (planned). It is the natural extension because the rule table already isolates exactly that residual set, so the judge's cost is paid only where rules cannot decide. *Red-team suite:* 60–100 cases covering prompt injection, forged or replayed tokens, wrong-owner IDs and races (planned). It is the natural next step because the executor's token, idempotency and recheck defences are the parts a hostile caller would target, and only adversarial cases can show whether they hold beyond our own synthetic worlds.

6. Novelty and Differentiation

The obvious baseline (the Starter Resources pattern) is a LangGraph agent that looks up a knowledge base and then acts. GroundGate differs in three ways: it treats UNVERIFIABLE as distinct from CONTRADICTED, it

spends lookups under an explicit budget, and it re-checks at commit time inside the executor rather than trusting the earlier verdict. We claim no first on any component. Deterministic state gates exist (Reddy et al., 2607.07405; LedgerAgent, 2606.20529; NOMOS, 2610.11030), as do policy-compiled guards (Zwerdling et al., 2507.16459). The closest work is AMV-Guard (2610.31446, concurrent), which picks memory premises to re-verify by bounded value of information, and SafeCommit (2608.04289), another budgeted-verification design. We study a different setting (a system of record behind a degradable cache), use a transparent rule table, and do not claim to beat it. Our contribution is an **evaluation**: business-API gating with four separated state views (agent-visible, verification-time, verifier-visible, mutation-time) while the evidence source degrades, comparing policies at an **equal lookup budget** and reporting where simple baselines win.

7. Evaluation Plan and Prototype Results

Design. 20 synthetic worlds × 500 actions × 8 source conditions (cache gaps, staleness, replica lag, outages, lookup budgets, drift). Parameters frozen on separate dev seeds; four error families held out; harm labelled from mutation-time state. **Baselines:** no guard, exact rules, closed-world cache, a fact gate reimplemented from NOMOS's *described* behaviour, always-query, GroundGate and three ablations. **Metrics:** harmful actions passing, false blocks, escalations, valid completions, lookups per action, expected loss; 95% bootstrap intervals over worlds.

Prototype findings (synthetic, not real model errors).

- Cache-first routing used 0.08–0.22 fewer lookups per action than always-query when the source was available, at equal gate-level safety.
- At 0.2 lookups per action, harmful calls passing fell from 19.5% to 10.3%, valid completions rose 10.6 points (CI 9.8–11.4) and valid-action escalations fell 10.7 points (59.2% to 48.5%). Residual harm remains: evidence quality bounds safety.
- A closed-world cache guard reaches 0% harm but completes only 61% of valid actions; at a false-block cost of 1 it often wins, at 20 GroundGate wins except in the ideal condition.
- In 30 multi-process races, naive check-then-act broke the refund invariant 30 times; the protected executor never did. Commit-time recheck reads the state that defines the label, so this is true by construction.

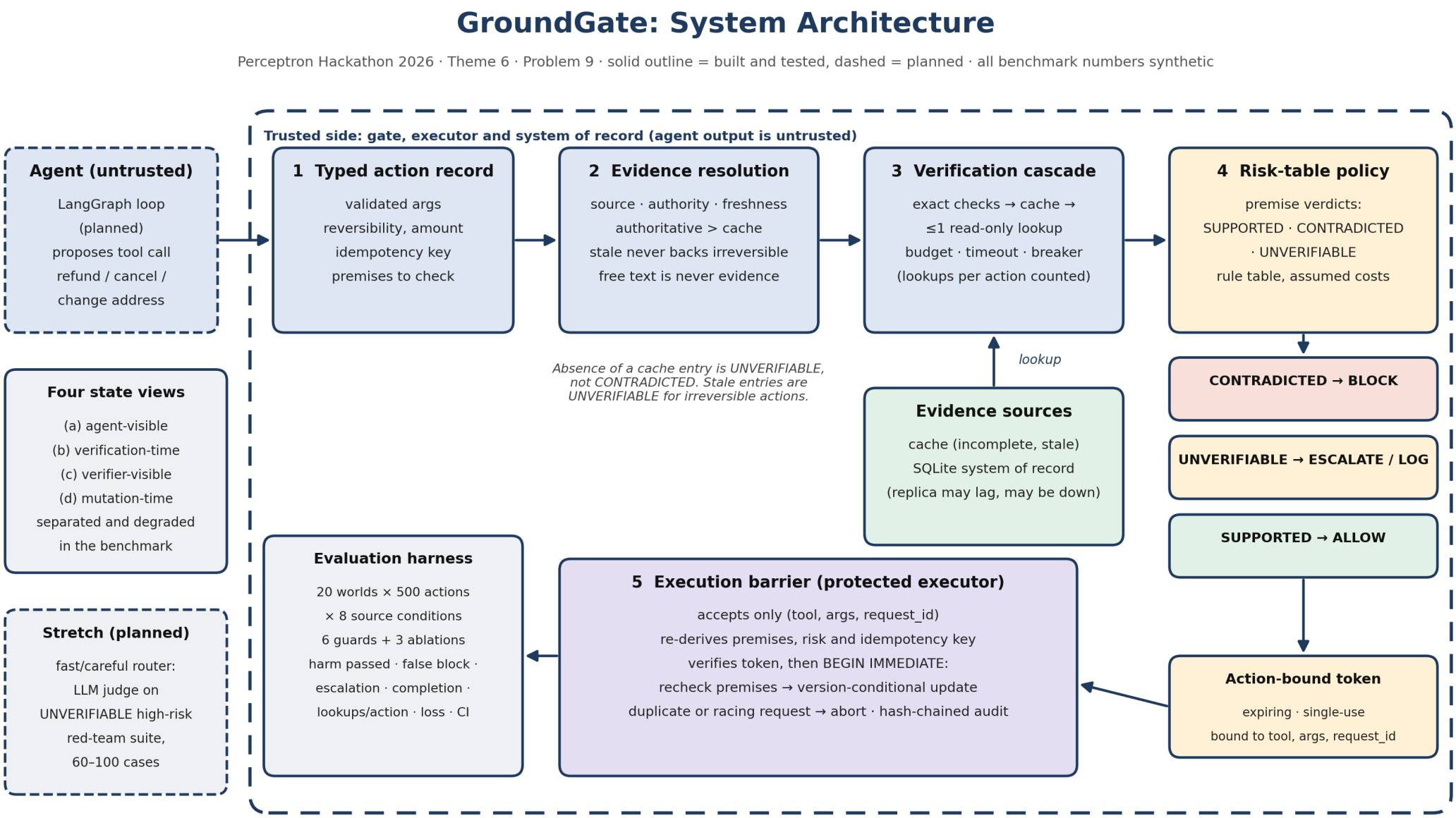
Planned, not done: a real-LLM agent on about 120 constructed scenarios (reported separately), 40–60 human-written cases by a teammate who did not build the gate, an LLM-judge baseline, and an executor view that lags the system of record.

8. Feasibility, Team Fit and Risks

Skills: [member 1: agent frameworks], [member 2: evaluation], [member 3: backend and security]. The core exists; remaining work is integration and evidence. **Plan:** to 9 Nov LangGraph agent; 10–20 Nov MVP demo; 21–30 Nov elicit real model errors, midway report; 1–14 Dec LLM judge, red-team suite, rehearsal. **Risks:** real model errors may be too rare to measure (reported as-is); synthetic circularity (stated); cost-dependent ranking (shown); scope growth (stretch items drop first).

References: arXiv 2607.07405; 2606.20529; 2610.11030; 2507.16459 (EMNLP 2025 Industry); 2610.31446; 2608.04289; AgentSpec 2503.18666; Progent 2504.11703; CaMeL 2503.18813.

Architecture and approach diagram



Trust boundary: only the executor and the system of record are trusted. Same-OS-privilege attackers are out of scope. Commit-time recheck reads the state that defines the harm label, so its 0% harm is true by construction.

Real-model errors, human-written cases, an LLM-judge baseline and a lagging executor view are planned, not yet done.